

Klasyfikacja danych przy użyciu algorytmu KNN.

Piotr Kardyński, numer indeksu: 119090, numer grupy: 3

1. Pobieranie danych, wstępna analiza danych wejściowych

```
def wczytaj_iris():  
    # x-cechy y-nazwy  
    iris = fetch_ucirepo(id=53)  
    X = iris.data.features  
    y = iris.data.targets  
    return X, y
```

Pobranie danych jest realizowane przez zmodyfikowany kod dostępny na <https://archive.ics.uci.edu/dataset/53/iris>. "X" to cechy czyli w tym przypadku fizyczne wymiary kwiatów.

W zbiorze Iris X ma 4 kolumny:

- sepal length (długość działki kielicha)
- sepal width (szerokość działki kielicha)
- petal length (długość płatka)
- petal width (szerokość płatka)

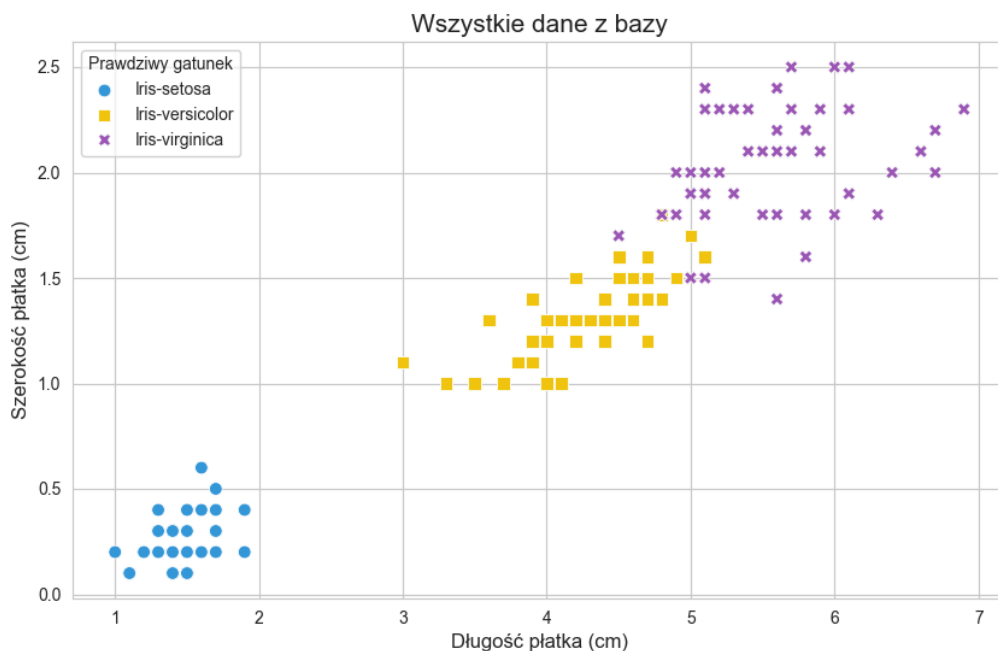
"y" to etykiety czyli nazwy gatunków przypisane do każdego z zestawów z "X", "y" zawiera w tym przypadku jedną z trzech wartości:

- Iris-setosa
- Iris-versicolor
- Iris-virginica

Każda z tych klas gatunków zawiera po 50 wystąpień w bazie.

2. Analiza danych

By lepiej poznać dane można je wyświetlić za pomocą wykresu (rysunek poniżej), rozróżniając je za pomocą klasy Iris-setosa, Iris-versicolor, Iris-virginica, oraz szerokością i długością płatka (ponieważ to je najbardziej odróżnia).



Dzięki temu wykresowi można zauważyć, że versicolor oraz virginica są podobne do siebie a klasa setosa jest całkowicie oddzielna.

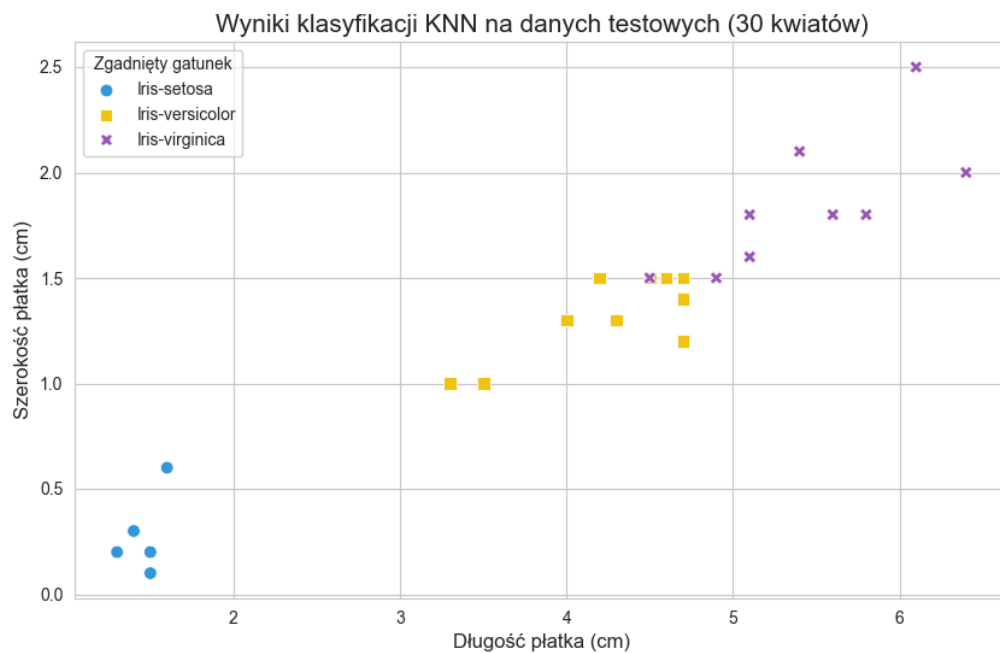
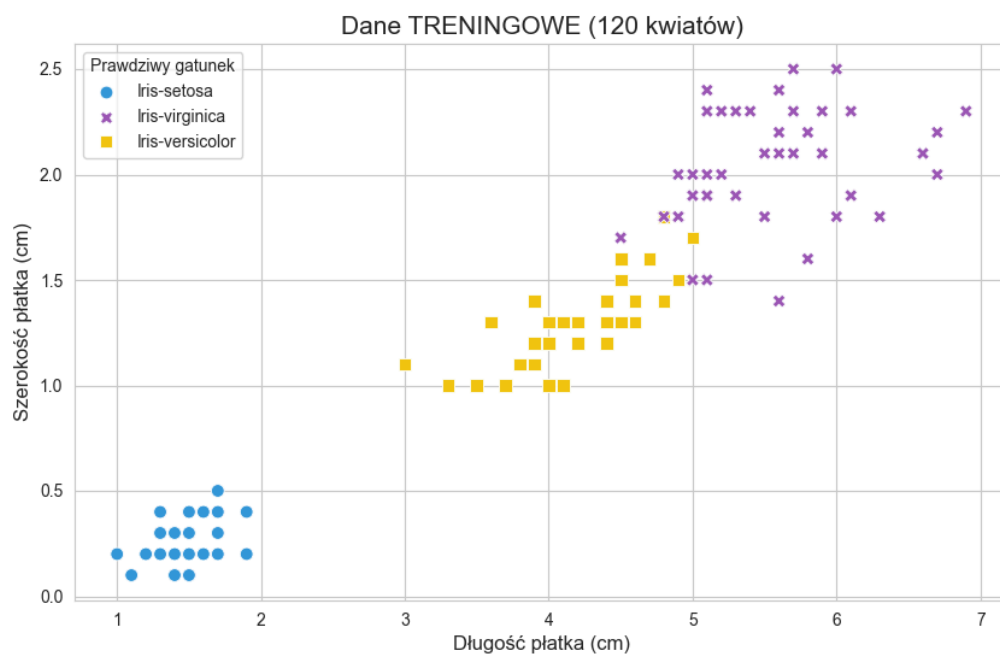
W dalszej części analizy dzielimy te dane na część do nauki (80%) i do testu (20%):

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=67)
```

Następnie tworzymy klasyfikator KNN, który "zapamiętuje" wzorce ze zbioru treningowego i na ich podstawie przewiduje gatunki dla ukrytych danych testowych, robi to porównując 3 najbliższych sąsiadów:

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train.values.ravel())
y_pred = knn.predict(X_test)
```

Kolejne wykresy przedstawiają: dane treningowe czyli 120 kwiatów wybranych losowo do treningu oraz wyniki klasyfikacji KNN dla 30 kwiatów które pozostały.



Klasa (Gatunek)	Precyzja (Precision)	Czułość (Recall)	F1-Score	Wsparcie (Support)
Iris-setosa	1.00	1.00	1.00	9

Iris-versicolor	1.00	0.80	0.89	15
Iris-virginica	0.67	1.00	0.80	6
Dokładność (Accuracy)			0.90	30
Macro Avg	0.89	0.93	0.90	30
Weighted Avg	0.93	0.90	0.90	30

Skuteczność algorytmu KNN: 90.00%

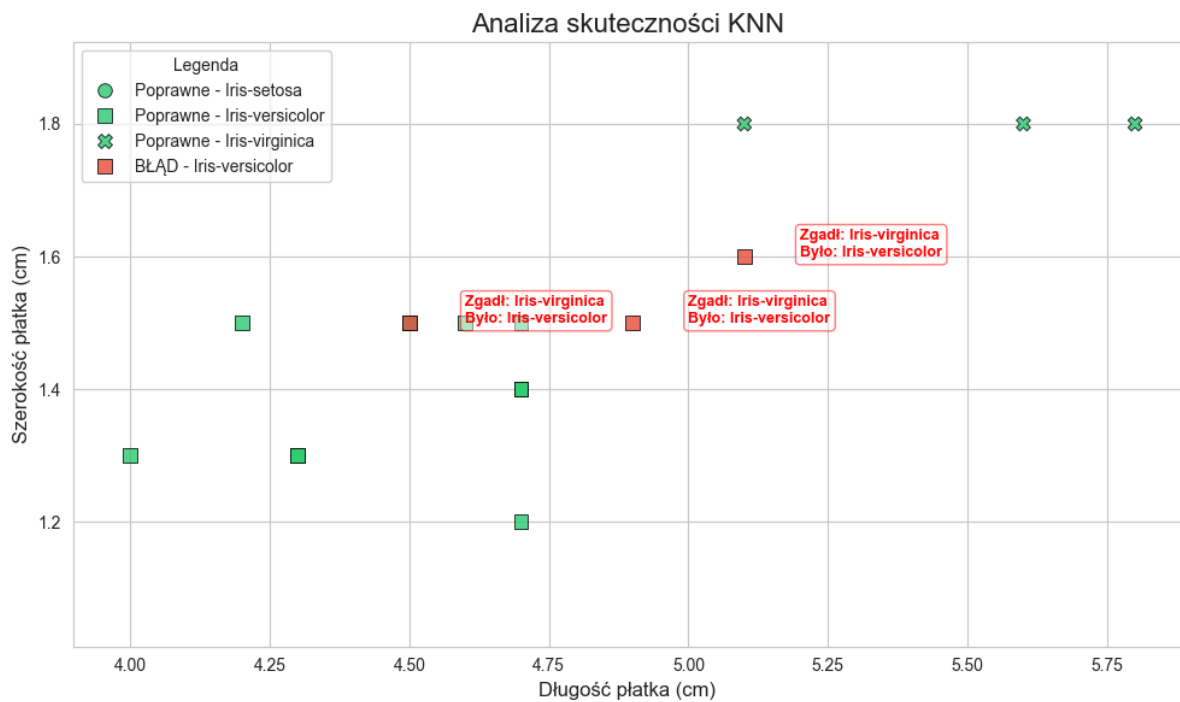
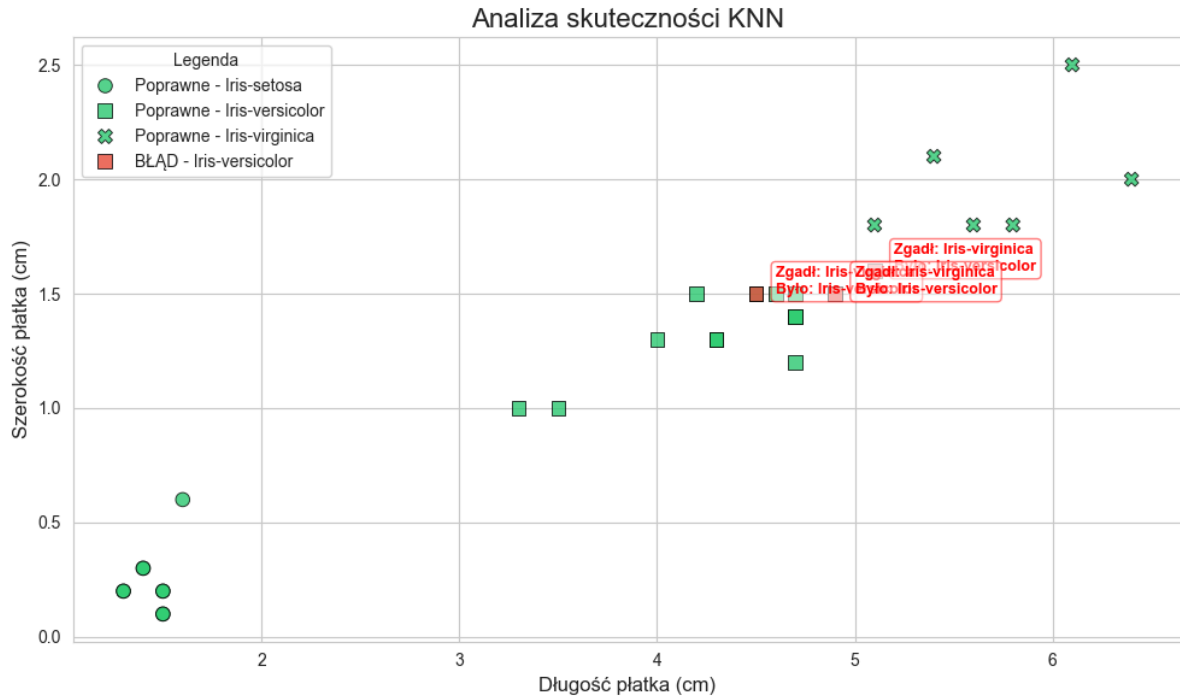
Precyzja (Precision): 88.89%

Czułość (Recall): 93.33%

F1-Score: 89.63%

Klasa	Co się stało?
Setosa	Model znalazł wszystkie (Recall 1.0) i nigdy się nie pomylił (Precision 1.0).
Versicolor	Model znalazł tylko 80% z nich (3 mu "uciekły"), ale te, które wskazał, były na 100% poprawne.
Virginica	Model znalazł wszystkie (Recall 1.0), ale „przy okazji” nazwał Virginicą też te 3 zgubione Versicolory (stąd słaba Precision 0.67).

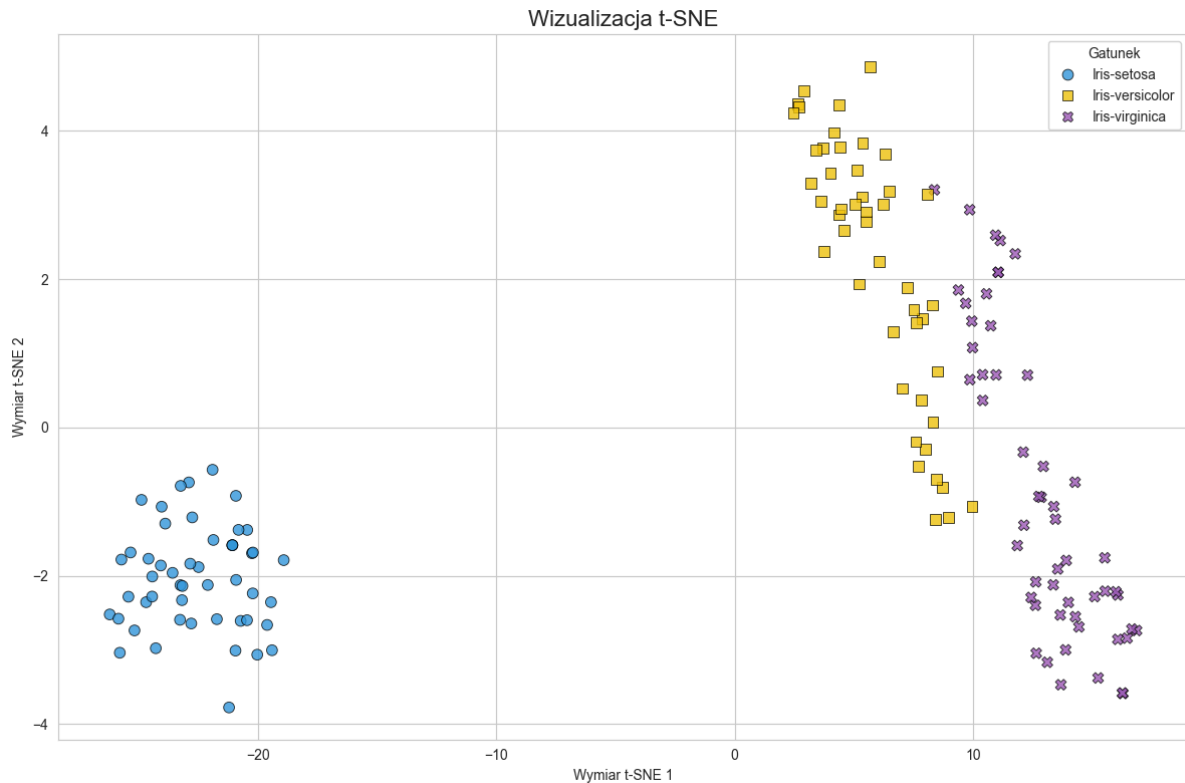
Poniżej przedstawiam wykres dobrych oraz złych trafień przez algorytm oraz miejsce w którym one się znajdują:



Dzięki tym wykresom można zauważyć, że błędy klasyfikacji pojawiają się tam gdzie klasa versicolor oraz virginica są ze sobą blisko według szerokości i długości płatka.

3. Wizualizacja przy wykorzystaniu TSNE

```
tsne = TSNE(n_components=2, random_state=67, perplexity=30)
```



Co widzimy:

1. Setosa (Niebieskie kółka): Ten gatunek znajduje się na kompletnym "odosobnieniu" po lewej stronie. Potwierdza to wyniki z raportu (Precision/Recall 1.0).
2. Relacja Versicolor vs. Virginica: Te dwa gatunki tworzą dwie "chmury" po prawej stronie, ale są one położone bardzo blisko siebie przez to dochodzi do błędów opisanych wcześniej.

Podsumowanie wizualizacji przy wykorzystaniu TSNE:

Wykres pokazuje oraz potwierdza skupiska klas gdy zostały one zredukowane przez algorytm z 4D do 2D. Klasa setosa jest oddzielna, a klasy versicolor i virginica mimo różnych nazw niektóre ich instancje są bardzo blisko siebie.